

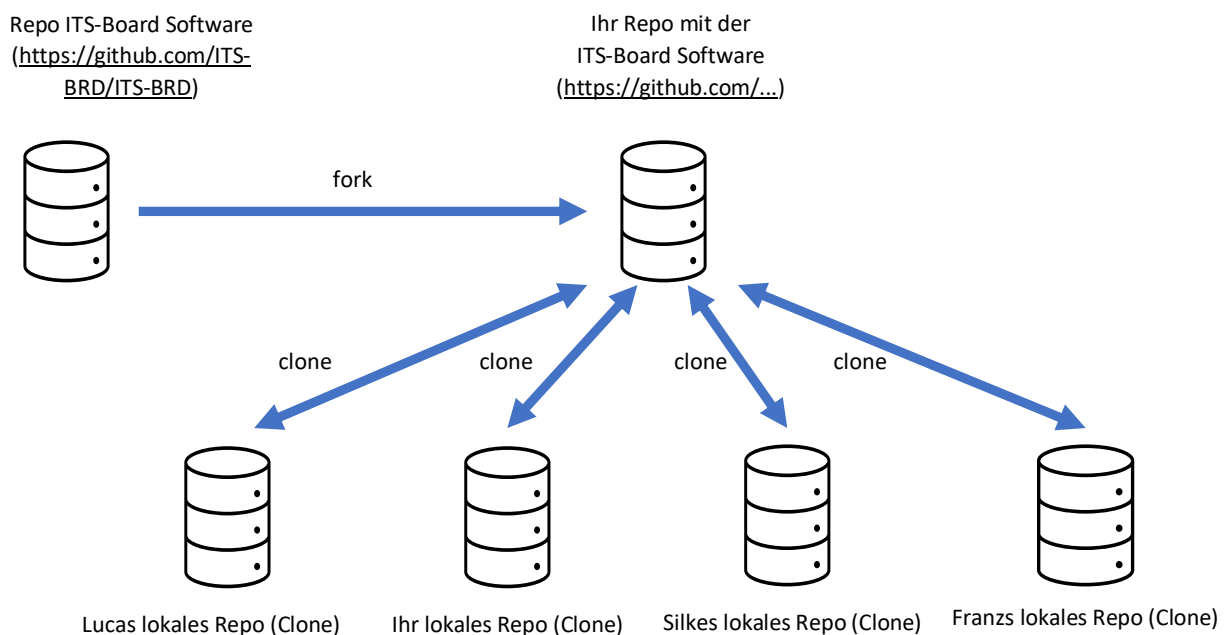
# Installation der Software zum ITS-Board

Zum ITS-Board gehört ein Software-Paket mit Basisfunktionalitäten, die Sie für die Entwicklung der Programme für das ITS-Board (teilweise) benötigen. Die Software ist in einem git Repository (Repo) zusammengefasst. git ist eine weit verbreitete Versionsverwaltungssoftware. Dieses Dokument ist eine Anleitung zur Installation des Software-Pakets zum ITS-Board auf Ihrem PC. Dabei wird git ohne Erklärung der benötigten git Funktionalitäten angewendet. Die Funktionsweise von git lernen Sie in anderen Veranstaltungen des Studiums.

## Hintergrund

Dieser Abschnitt gibt Ihnen Hintergrundinformationen, so dass Sie ein grobes Verständnis bezüglich des Installationsprozesses gewinnen. Daher fokussiert er sich so knapp wie möglich auf die für den Installationsprozess relevanten Zusammenhänge und lässt somit viele Punkte offen.

git verwaltet die Software in sogenannten Repositories (Repos). Ein Repo bündelt die Dateien eines Softwarepakets. Repos können sowohl auf einen Server in Netz liegen als auch auf Ihrem lokalen Rechner. Folgendes Bild stellt die Relation zwischen den einzelnen Repos, die hier benötigt bzw. erstellt werden, dar.



Die Software zum ITS-Board steht als Repo auf dem github Server zur Verfügung (Repo links oben im Bild). Von diesem Repo erstellen Sie einen Fork (in 0-ter Näherung eine Kopie) und erhalten somit Ihr eigenes Repo mit der Software zum ITS-Board. Dieses Repo liegt ebenfalls auf dem github Server. Damit Sie mit der Software arbeiten können, müssen Sie Ihr Repo auf dem github Server in ein lokales Repo auf Ihrem Rechner klonen (s. untere Zeile von Repos im Bild). Ein geklontes lokales Repo ist eine besondere Kopie des Repos auf dem github Server. Das Besondere ist, dass Sie Änderungen, die Sie in Ihrem lokalen Repo gemacht haben, wieder in das Repo auf dem Server zurückspielen können. Praktikumpartner\*innen (im Bild mit dem Namen Luca) können ebenfalls Ihr Repo auf dem github Server klonen. Auf diesem Weg kann Luca zum Beispiel Änderungen im lokalen Repo vornehmen und diese auf Ihr Repo auf dem Server zurückspielen (push). Sie können dann diese Änderungen in Ihren lokalen Clone des Repos einspielen – mit Ihren Änderungen mischen (pull bzw. merge). Die Praktikumsbetreuer\*innen können ebenfalls Ihr Repo auf dem github Server klonen und somit sehen,

wie der Stand Ihrer Arbeiten ist und auf Wunsch helfen. Über diesen Weg geben Sie auch Ihre Praktikumsaufgaben ab.

## Erstellung eines github Accounts

Sie benötigen einen github Account. Wenn Sie noch keinen Account haben, können Sie diesen über <https://github.com/> erstellen.

## Fork des Repos der ITS-Board Software

Sie benötigen entsprechende Zugriffsrechte zur Erstellung eines „fork“ des Repos der ITS-Board Software.

- Tragen Sie zum Erhalt der Zugriffsrechte in folgende dfn Buchungsliste Ihren **github Benutzernamen (username)** ein.

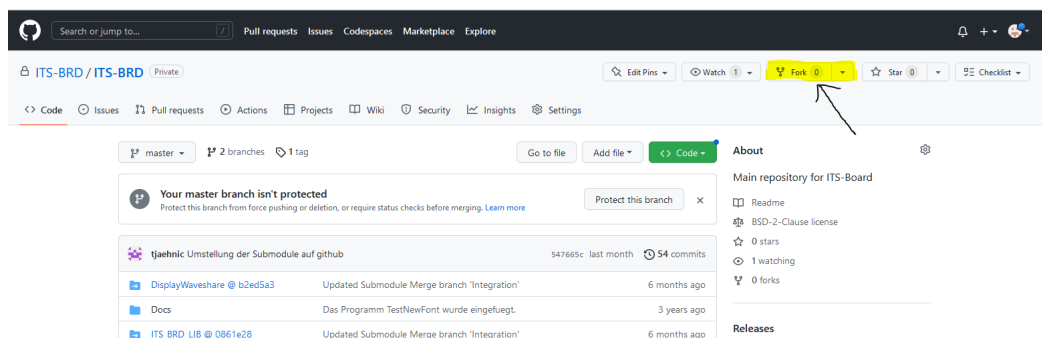
<https://terminplaner6.dfn.de/b/1d425072b48e16d4aa868bd1960176cb-163814>

- **Nachdem** das Repo für Ihren github Benutzernamen freigegeben wurde, kann mit dem nächsten Schritt fortgefahren werden.

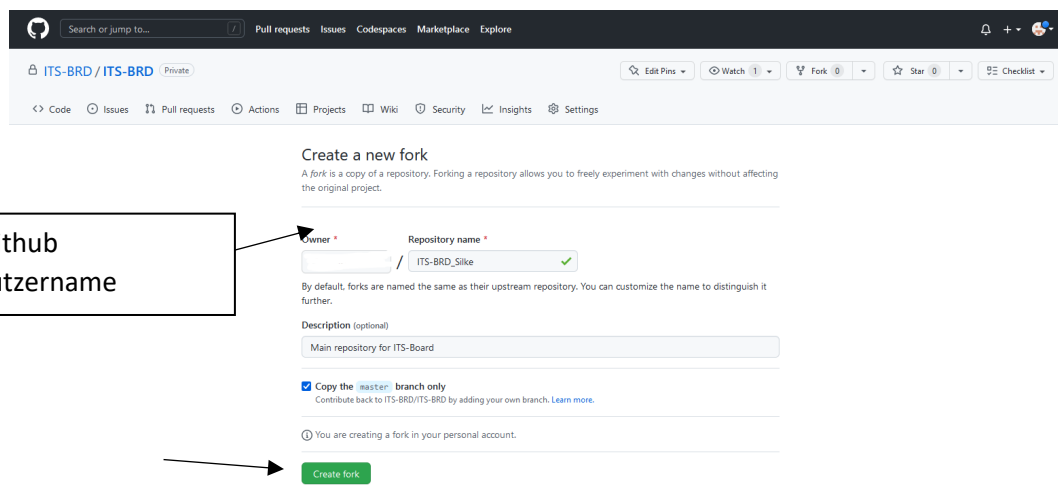
•

Nun wird ein Fork des Repos der ITS-Board Software erstellt (s. linke obere Ecke des Bilds der ersten Seite). Dieser Fork (diese Kopie) liegt dann auf dem github Server unter Ihrem Account.

- Gehen Sie auf den link <https://github.com/ITS-BRD/ITS-BRD>. (ggf. müssen Sie sich erneut bei github.com anmelden.) Nun sind Sie auf der Seite des github Servers, die das Repo der ITS-Board Software enthält. Drücken Sie auf den „Fork“ Button zur Erstellung Ihres Forks von diesem Repo.



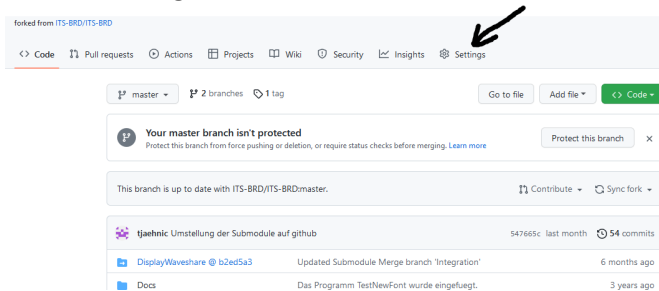
- Füllen Sie nun die Maske zur Erstellung des Forks aus (Ihr github Benutzernamen und der Namen des Repos, das Ihren Fork der ITS-Board Software enthält).
- Erstellen Sie Ihren den Fork über den Button „Create fork“.



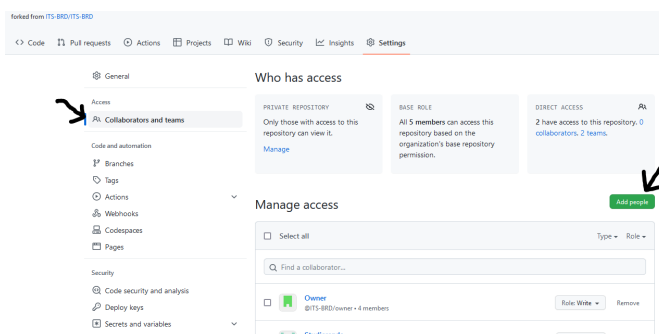
## Zugriffsrechte für Ihr Repo auf dem github Server einstellen

Wie auf dem Bild von der ersten Seite ersichtlich, greifen unterschiedliche Personen auf Ihr Repo auf dem github Server zu. Dieser Abschnitt beschreibt die Gewährung der Zugriffsrechte für diese Personen.

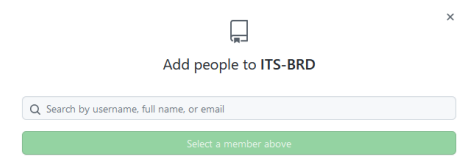
- Gehen Sie auf den Link zur Ihrem Repo auf den github Server. Wenn Sie nicht auf der entsprechenden Seite sind, klicken Sie auf die schwarze Katze links oben in der github GUI. Dann sollten Sie auf der linken Seite die von ihnen zuletzt benutzten Repos sehen und somit auch Ihr Repo auf dem Github Server. Wählen Sie das Repo aus.
- Wählen Sie nun Settings aus.



- Wählen Sie links auf der Seite „Collaborators and teams“ aus.



- Über „Add people“ Button können Sie nun Personen Zugriffsrechte erteilen.



- Geben Sie den Betreuer\*innen (github Benutzernamen **SilkeBehn** und **FranzKorf**) **lesende** Zugriffsrechte.
- Erfragen Sie den github Benutzernamen Ihrer Praktikumpartner\*in. Vergeben Sie **schreibende** Zugriffsrechte. Wenn Sie noch keinen Praktikumpartner\*in haben, können Sie diesen Schritt jederzeit nachholen.

## Erstellung eines Tokens für den Zugriff auf Ihr Repo

Ihr eigenes Repo der ITS-Board Software liegt auf dem github Server im Netz und nicht lokal auf Ihrem PC (s. Bild auf der ersten Seite). Damit Sie mit der Software arbeiten können, müssen Sie das Repo auf dem github Server in ein lokales Repo auf Ihrem Rechner klonen. Wann immer Sie von Ihrem Rechner aus auf Ihr Repo auf dem github Server zugreifen wollen, müssten Sie sich mit Ihrem Benutzernamen und Passwort anmelden. Das ist sehr umständlich und führt schnell dazu, dass man Sicherheitsaspekte

wie zum Beispiel sichere Passwörter vernachlässigt. Mit einem SSL-Schlüssel oder einen Token zur Authentifizierung umgehen Sie den ermüdenden Anmeldeprozess und stellen trotzdem sicher, dass nur Sie auf Ihren github Account zugreifen. Wir verwenden hier ein Token.

In <https://docs.github.com/de/authentication/keeping-your-account-and-data-secure/creating-a-personal-access-token> beschreibt der Abschnitt “Creating a personal access token (classic)” das Erzeugen eines Tokens.

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

GitHub Token

What's this token for?

Expiration \*

90 days The token will expire on Thu, Jun 8 2023

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

|                                                     |                                                                     |
|-----------------------------------------------------|---------------------------------------------------------------------|
| <input checked="" type="checkbox"/> repo            | Full control of private repositories                                |
| <input checked="" type="checkbox"/> repo:status     | Access commit status                                                |
| <input checked="" type="checkbox"/> repo:deployment | Access deployment status                                            |
| <input checked="" type="checkbox"/> public_repo     | Access public repositories                                          |
| <input checked="" type="checkbox"/> repo:invite     | Access repository invitations                                       |
| <input checked="" type="checkbox"/> security_events | Read and write security events                                      |
| <input type="checkbox"/> workflow                   | Update GitHub Action workflows                                      |
| <input checked="" type="checkbox"/> write:packages  | Upload packages to GitHub Package Registry                          |
| <input checked="" type="checkbox"/> read:packages   | Download packages from GitHub Package Registry                      |
| <input type="checkbox"/> delete:packages            | Delete packages from GitHub Package Registry                        |
| <input type="checkbox"/> admin:org                  | Full control of orgs and teams; read and write org projects         |
| <input checked="" type="checkbox"/> write:org       | Read and write org and team membership; read and write org projects |
| <input type="checkbox"/> read:org                   | Read org and team membership; read org projects                     |
| <input type="checkbox"/> manage_runners:org         | Manage org runners and runner groups                                |
| <input type="checkbox"/> admin:public_key           | Full control of user public keys                                    |
| <input type="checkbox"/> write:public_key           | Write user public keys                                              |
| <input type="checkbox"/> read:public_key            | Read user public keys                                               |
| <input checked="" type="checkbox"/> admin:repo_hook | Full control of repository hooks                                    |
| <input checked="" type="checkbox"/> write:repo_hook | Write repository hooks                                              |
| <input checked="" type="checkbox"/> read:repo_hook  | Read repository hooks                                               |
| <input type="checkbox"/> admin:org_hook             | Full control of organization hooks                                  |
| <input type="checkbox"/> gist                       | Create gists                                                        |
| <input type="checkbox"/> notifications              | Access notifications                                                |
| <input type="checkbox"/> user                       | Update ALL user data                                                |
| <input type="checkbox"/> read:user                  | Read ALL user profile data                                          |
| <input type="checkbox"/> user:email                 | Access user email addresses (read-only)                             |
| <input type="checkbox"/> user:follow                | Follow and unfollow users                                           |
| <input checked="" type="checkbox"/> delete:repo     | Delete repositories                                                 |
| <input type="checkbox"/> write:discussion           | Read and write team discussions                                     |
| <input type="checkbox"/> read:discussion            | Read team discussions                                               |
| <input type="checkbox"/> admin:enterprise           | Full control of enterprises                                         |
| <input type="checkbox"/> manage_runners:enterprise  | Manage enterprise runners and runner groups                         |
| <input type="checkbox"/> manage_billing:enterprise  | Read and write enterprise billing data                              |
| <input type="checkbox"/> read:enterprise            | Read enterprise profile data                                        |
| <input type="checkbox"/> audit_log                  | Full control of audit log                                           |
| <input type="checkbox"/> read:audit_log             | Read access of audit log                                            |
| <input type="checkbox"/> codespace                  | Full control of codespaces                                          |
| <input type="checkbox"/> codespace:secrets          | Ability to create, read, update, and delete codespace secrets       |
| <input type="checkbox"/> project                    | Full control of projects                                            |
| <input type="checkbox"/> read:project               | Read access of projects                                             |
| <input type="checkbox"/> admin:gpg_key              | Full control of public user GPG keys                                |
| <input type="checkbox"/> write:gpg_key              | Write public user GPG keys                                          |
| <input type="checkbox"/> read:gpg_key               | Read public user GPG keys                                           |
| <input type="checkbox"/> admin:ssh_signing_key      | Full control of public user SSH signing keys                        |
| <input type="checkbox"/> write:ssh_signing_key      | Write public user SSH signing keys                                  |
| <input type="checkbox"/> read:ssh_signing_key       | Read public user SSH signing keys                                   |

Generate token Cancel

## Der Token wird generiert.

Settings / Developer settings

GitHub Apps  
OAuth Apps  
Personal access tokens  
Fine-grained tokens  
Tokens (classic)

Personal access tokens (classic)

Generate new token \* Revoke all

Tokens you have generated that can be used to access the GitHub API.

Make sure to copy your personal access token now. You won't be able to see it again!

✓ [REDACTED] Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Hier kann man sich das Token in die Zwischenablage kopieren. Sie brauchen es später noch. Ein Token sollte wie ein Passwort geschützt werden.

## Installation der gitbash auf Ihrem PC

Damit git unter Windows von der Kommandozeile aus nutzbar ist – also git Befehle in Textform eingegeben werden können, muss die gitbash von <https://git-scm.com/download/win> z.B. als git für Windows Portable heruntergeladen werden.

## Ein lokales Repo erstellen

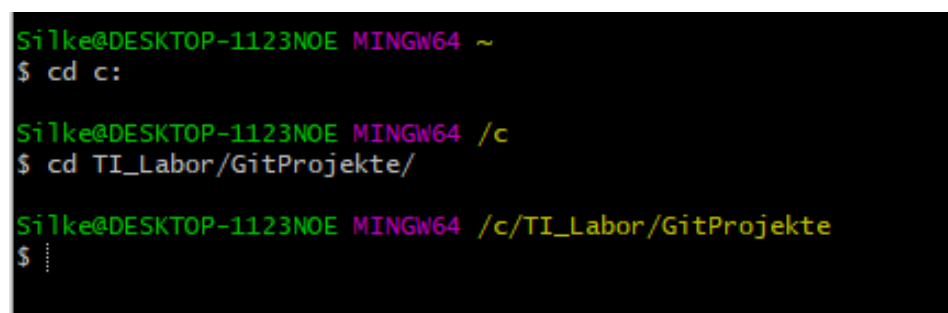
Nun muss noch ein lokales Repo von Ihrem Repo aus dem github geklont werden. Dies geschieht in der gitbash.

- In Windows einen Ordner für die Git Projekte anlegen, z.B. C:\TI\_Labor\gitProjekte
- Die gitbash starten (git-bash.exe)



```
MINGW64:/c/Users/Silke
Silke@DESKTOP-1123NOE MINGW64 ~
$ |
```

- In den vorher erstellten Ordner mit folgender Befehlsfolge wechseln
  - `cd c:`
  - `cd TI_Labor/gitProjekte`

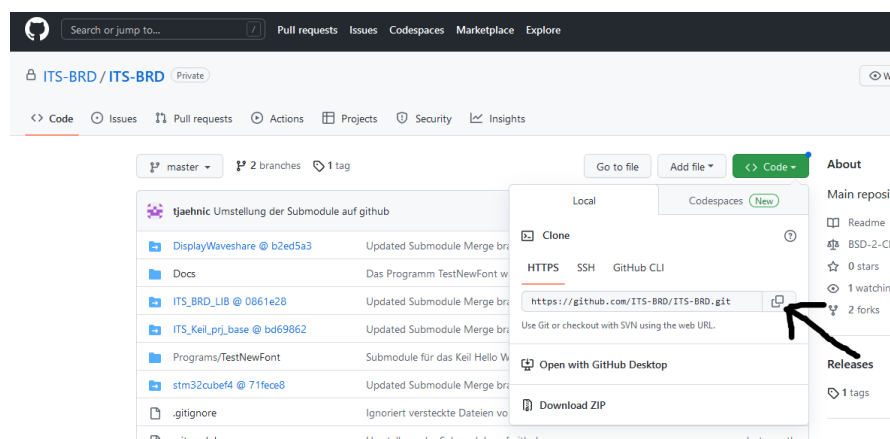


```
Silke@DESKTOP-1123NOE MINGW64 ~
$ cd c:

Silke@DESKTOP-1123NOE MINGW64 /c
$ cd TI_Labor/GitProjekte/

Silke@DESKTOP-1123NOE MINGW64 /c/TI_Labor/GitProjekte
$
```

- Zum Klonen Ihres git Repo auf dem github Server wird der Link auf dieses Repo wie folgt in die Zwischenablage kopiert:
  - Gehen Sie auf den Link zur Ihrem Repo aus den github Server (also in Ihrem Internet-Browser).
  - Drücken Sie den grünen Code-Button.
  - Kopieren Sie den **HTTPS** Link des Repos mit dem Button  in die Zwischenablage.



- In der gitbash wird Ihr Repo auf dem github Server über folgendem Befehl auf Ihren Rechner geklont.

```
git clone --recursive <Link auf Ihr Repo aus der Zwischenablage> ITSBoard
```

**Achtung:** Nun werden github Benutzername und Passwort abgefragt. Statt des Passworts geben Sie das oben erzeugte und in der Zwischenablage gespeicherte **Token** ein.

Jetzt sollten im Verzeichnis `c:\TI-Labor\gitProjekte\ITSBoard` die Daten zu Ihrem lokalen Repo liegen.

Herzlichen Glückwunsch, bei den Installationsaufgaben für GT und GTP haben Sie einen großen Schritt gemacht. Nun können Sie Ihr eigenes Keil Projekt erstellen und die erste Praktikumsaufgabe bearbeiten.

In der Datei „Keil Projekt erstellen.pdf“ finden Sie eine Beschreibung zur Erstellung eines Keil Projekts – was sehr einfach ist.